

Fabric Flo — Finish & Submit Checklist

(Windows-aware)

Everything left to get Fabric Flo onto the Apple App Store and Google Play — from a Windows 11 PC. Generated for the operator · June 2026 · Not legal advice

App identity (already set): **App name** Fabric Flo · **Bundle/App ID** `app.fabricflo.tracker` · **Version** 1.0.0 (build/versionCode 1).

*Brand new to this? Read **docs/START_HERE.md** first — it's the simplest, no-jargon overview. This checklist is the detailed version of that roadmap.*

Related docs: `docs/TRANSFER_TO_WINDOWS.md` (move to Windows),
`docs/APP_STORE_RELEASE_CHECKLIST.md` (full reference),
`docs/STORE_LISTING.md` (listing copy),
`docs/APP_REVIEW_NOTES.md` (reviewer notes),
`docs/BACKEND_SETUP.md` (Supabase),
`docs/INVESTOR_NETLIFY.md` (web host).

The one thing to understand about Apple on Windows

Windows **cannot** build an iOS app — only Apple computers can. You have three ways to get an iOS build **without buying a Mac**:

Option	Mac needed?	Best for
Codemagic (cloud Mac CI)	❌ No	Recommended — builds from GitHub, can auto-submit to Apple
Friend's MacBook Air	✅ Theirs	Occasional manual builds; they <code>git clone</code> your repo (no USB)
Rent a cloud Mac (MacinCloud, etc.)	❌ No	If you prefer remote-desktop into a real Mac

Android builds run fine **on Windows** with Android Studio. So a realistic plan is: **Windows for Android + Codemagic for iOS**. This repo includes a ready-made

codemagic.yaml .

Phase 0 — Foundations (do once, from Windows)

- ☐ Project running on Windows — see `docs/TRANSFER_TO_WINDOWS.md` (`npm install` , `npm run dev`).
 - ☐ `.env` recreated with your Supabase URL/key and public URL.
 - ☐ Code is pushed to GitHub (`git push` works).  *Already done* — *repo is live*.
-

Phase 1 — Live backend + public website (required before either store)

Both stores require a **public HTTPS Privacy Policy URL**, and password reset needs a live site.

- ☐ **Supabase production project** created.
 - ☐ Run migrations `001 – 009` (paste `supabase/APPLY_ALL_MIGRATIONS.sql` from `npm run supabase:bundle`).
 - ☐ **Deploy the web app** (Netlify) — connect the GitHub repo at <https://app.netlify.com> (auto-builds on every push).
 - ☐ Set Netlify **environment variables** and redeploy:
 - `VITE_SUPABASE_URL` , `VITE_SUPABASE_ANON_KEY`
 - `VITE_FABRIC_FLO_BACKEND=normalized`
 - `VITE_PUBLIC_APP_URL` (your live URL, no trailing slash)
 - `VITE_SUPPORT_EMAIL` , `VITE_PRIVACY_EMAIL`
 - ☐ **Supabase Auth URLs** match the Netlify domain.
 - ☐ These URLs load: `/` , `/app` , `/privacy` , `/terms` .
 - ☐ `npm run verify:release` passes.
-

Phase 2 — Store accounts (do early; approvals take time)

- ☐ **Apple Developer Program** — \$99/year — <https://developer.apple.com> (enroll as your business/individual).
 - ☐ **Google Play Console** — \$25 one-time — <https://play.google.com/console>.
 - ☐ Decide listing details with `docs/STORE_LISTING.md` (name, subtitle, description, keywords, category).
 - ☐ Create a **demo account** on your live backend for reviewers (see `docs/APP_REVIEW_NOTES.md`).
-

Phase 3 — Android (build on Windows)

You can do this entirely on your Windows 11 PC with **Android Studio**.

- ☐ Install Android Studio (see `docs/TRANSFER_TO_WINDOWS.md`).
- ☐ Build + sync the web assets into the native shell:

```
npm run build
npx cap sync android
npm run cap:android      # opens Android Studio
```

- ☐ In Android Studio: **Build** ▶ **Generate Signed App Bundle / APK** ▶ **Android App Bundle (.aab)**.
- ☐ **Create an upload keystore** when prompted — **back it up somewhere safe** (losing it means you can't update the app later). Store the passwords in a password manager.
- ☐ Confirm `versionCode 1 / versionName "1.0.0"` in `android/app/build.gradle` (bump for each update).
- ☐ In Play Console: create the app → **Internal testing** track → upload the `.aab` .
- ☐ Fill **Data safety** form (email, inventory/scan content, camera for QR; no tracking in v1).
- ☐ Complete **content rating** questionnaire.
- ☐ Add **Privacy Policy URL** = `https://YOUR_DOMAIN/privacy` .
- ☐ Add screenshots + feature graphic.

- ☐ Promote Internal testing → **Production** (staged rollout recommended).

Phase 4 — iOS via Codemagic (no Mac required)

This repo includes `codemagic.yaml` with an `ios-release` workflow. For the exact keys and variables to paste in, use the fill-in-the-blank sheet

docs/CODEMAGIC_SETUP.md (PDF:

`docs/Fabric_Flo_Codemagic_Setup.pdf`).

- ☐ Sign in at <https://codemagic.io> with **GitHub** and authorize the `fabric-flo` repo.
- ☐ In **Codemagic** ▶ **Teams** ▶ **Integrations**, connect your **Apple Developer / App Store Connect API key** (App Store Connect → Users and Access → Integrations → App Store Connect API → generate a key; upload the `.p8` , Key ID, and Issuer ID to Codemagic).
- ☐ In App Store Connect, create the app record with bundle ID `app.fabricflo.tracker` .
- ☐ Set Codemagic environment variables (group `appstore`): `APP_STORE_CONNECT_*` per the comments in `codemagic.yaml` , plus your web env (`VITE_*`) so the build embeds the right backend.
- ☐ Trigger the `ios-release` workflow → Codemagic builds the `.ipa` on a cloud Mac and uploads to **TestFlight**.
- ☐ Test via **TestFlight** on a real iPhone (scan real QR + rental labels; offline → online sync).
- ☐ In App Store Connect: complete listing, **App Privacy** labels, **Age rating** (typically 4+), Privacy Policy URL, screenshots → **Submit for Review**.
- ☐ Paste reviewer notes + demo login from `docs/APP_REVIEW_NOTES.md` .

*Prefer a real Mac? Same result by hand: on a Mac run `npm run build:native` , then in Xcode **Product** ▶ **Archive** ▶ **Distribute App** ▶ **App Store Connect**.*

Phase 5 — On-set / device testing (before going live)

- ☐ TestFlight (iOS) + Play internal testing (Android) with real crew.
 - ☐ Offline test: airplane mode → scan → place → reconnect → sync completes.
 - ☐ Rental list & log: **Download** (CSV + PDF) and **Upload** on a phone.
 - ☐ Account deletion + "delete data on device".
 - ☐ Forgot-password email (needs correct `VITE_PUBLIC_APP_URL`).
-

Phase 6 — After launch

- ☐ Monitor Supabase usage, errors, backups.
 - ☐ For each update: fix → bump version
(`versionName` / `MARKETING_VERSION` + `versionCode` /build) → `npm run build` → `cap sync` → rebuild (Android Studio / Codemagic) → resubmit.
 - ☐ Periodic `npm audit` .
 - ☐ Update Privacy Policy if you add analytics or new data collection.
-

Quick "what's done vs. what's left"

Area	Done in repo	You still do
Code on GitHub	✅ live at <code>KeatingMichael/fabric-flo</code>	Keep pushing changes
App icons + splashes	✅ generated (iOS/Android/PWA)	—
Version numbers	✅ 1.0.0 / build 1	Bump per update
Listing copy & reviewer notes	✅ drafted	Fill domain/email, capture screenshots, make demo login
Cloud build config	✅ <code>codemagic.yaml</code>	Connect Apple key + run workflow
Live backend + website	docs ready	Create Supabase + Netlify, set env, deploy

Store accounts	—	Apple (\$99/yr) + Google (\$25)
Signing	—	Android keystore (back up!) + Apple key in Codemagic

Commands you'll use most (work on Windows)

```
npm install                # first time on a machine
npm run dev                # run locally
npm run build              # production web build
npx cap sync android      # push web build into the Android
shell
npm run cap:android        # open Android Studio
npm run netlify:deploy     # deploy the website/PWA
npm run verify:release     # pre-release sanity checks
npm run guides:pdf         # regenerate the Windows + submit
PDFs
```

Fabric Flo — Film fabric & bag tracker for productions. Repo:

<https://github.com/KeatingMichael/fabric-flo>